Chapter 5

IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter presents the implementation and testing of the Marrybrown Sales and Payment Analytics Platform. Whereas Chapter 4 described the analysis and design of the platform, this chapter records how the implemented platform was developed across the reporting database and ETL layer, the FastAPI semantic/API layer, and the React-based portal layer. It also documents the validation and testing activities used to confirm report accuracy, user-facing behaviour, and internal processing reliability.

To keep the chapter academically appropriate and public-safe, implementation details are presented at technical level without exposing credentials, secret configurations, or operationally sensitive access instructions.

5.2 System Development

The implemented platform was developed as a multi-layer internal reporting system intended to reduce operational dependence on a vendor-managed reporting portal while preserving report traceability and validation discipline. The completed implementation consists of three primary layers. First, selected source-system data were replicated into a company-managed Microsoft SQL Server reporting database through controlled ETL processes. Second, report logic was reconstructed in a FastAPI backend that acted as a semantic/API layer over the replicated data. Third, a React-based reporting portal was implemented to provide authenticated report access, filtering, export functions, and selected administrative tools.

The development process followed the iterative and incremental methodology described in Chapter 3. This was necessary because several report modules could not be treated as straightforward query transcription tasks. In practice, report development involved repeated cycles of source-data tracing, semantic rule reconstruction, mismatch investigation, validation against vendor outputs, and refinement of the reporting workflow in both backend and frontend layers.

5.2.1 Reporting Database and ETL Development

The reporting-database implementation centred on a company-managed Microsoft SQL Server environment that preserved selected source tables through a 1:1 replication strategy. This replica design was chosen to retain source fidelity for parity validation and reconciliation, while allowing report logic to be rebuilt independently of the external portal. Rather than flattening all logic into the database layer, the implementation preserved transactional and reference data close to source structure and left report-specific grouping, derivation, and presentation logic to the semantic/API layer.

The ETL implementation did not rely on a single loading path. Instead, it evolved into a controlled replication model with separate but coordinated operational modes. Manual utilities were retained for broad baseline loading, historical backfill, and targeted recovery work, especially for dated tables that required replay across defined time windows. In parallel, a staged daily ETL orchestrator was implemented for routine dated-table maintenance and reference-data refreshes. This staged design used boundary-based replacement logic so that repeated execution of the same reporting window could safely refresh the replica without introducing duplicate or drifting rows.

In the final implemented state, the ETL layer also supported portal-triggered manual sync requests and scheduled automation. Portal-triggered sync allowed authorised administrators to request dated-table or reference-table refreshes from the portal without executing ETL commands directly on the database host. Scheduled ETL

automation was also implemented as an operational feature for unattended daily dated refreshes and nightly retained-reference refreshes, with administration performed through the platform's automation-management workflow. This arrangement gave the platform both controlled manual recovery capability and operational automation capability, without requiring ordinary report users to interact with the ETL layer directly.

5.2.2 Backend API Development

The backend was implemented using FastAPI and functioned as the semantic/API layer of the platform. Its principal role was to reconstruct vendor-aligned report logic over the replicated database and expose the resulting report outputs through stable, parameter-driven endpoints. This design avoided direct frontend dependence on raw database structures and made it possible to keep report logic versioned and maintainable within the backend layer.

Backend development involved several recurring tasks across report modules. These included request-parameter definition, outlet and date-range filtering, shared handling of sales-status or payment-status filters where relevant, structured row output, and export-oriented response shaping. Reports that required grouped behaviour in the portal also needed backend outputs that could be normalised reliably for frontend grouping, subtotal rendering, advanced search, and export generation. In this way, the backend was not merely a database wrapper; it became the primary location where report-specific business rules, derived fields, and stable response contracts were implemented.

The backend layer also supported non-report operational functions required by the completed system. These included authentication, admin-managed user functions, portal-triggered sync control, automation-management support, and quality-check workflows linked to selected sync operations. Although these supporting features were not the central academic contribution of the project, they were necessary to make the implemented platform operationally usable in a real company environment.

5.2.3 Frontend Portal Development

The frontend portal was implemented using React and was designed to provide authenticated internal access to reports through a workflow-oriented rather than dashboard-oriented interface. The implemented report experience followed a common pattern: users sign in, access the report catalogue, select a report, define parameters, query the report, review the resulting table and totals, and export the output where needed. This structure was intentionally aligned with the practical behaviour expected by Finance and Operations users, especially for reconciliation and operational checking tasks.

Frontend development relied on shared report architecture rather than isolated page-specific implementations. A report registry, shared page shell, shared filters panel, grouped-table component, export toolbar, and supporting hooks were reused across report modules. This reduced duplication and made it feasible to add new formal-scope reports and additional customised reports through a consistent frontend pattern. The report-list view, report-query view, grouped-table behaviour, advanced search support, and export actions were therefore developed as reusable platform capabilities rather than as unrelated page fragments.

This user-facing reporting pattern is reflected in Figure 5.1 through a sanitised composite view that combines the Reports Hub with a representative report-query page. The figure represents the actual portal flow retained in the implementation: users begin from the catalogue of available reports, move into the selected module to define the required parameters, and then review grouped output through the shared query interface. Figure 5.1 therefore shows that the portal was developed as a consistent report-execution surface rather than as a collection of unrelated dashboard screens.

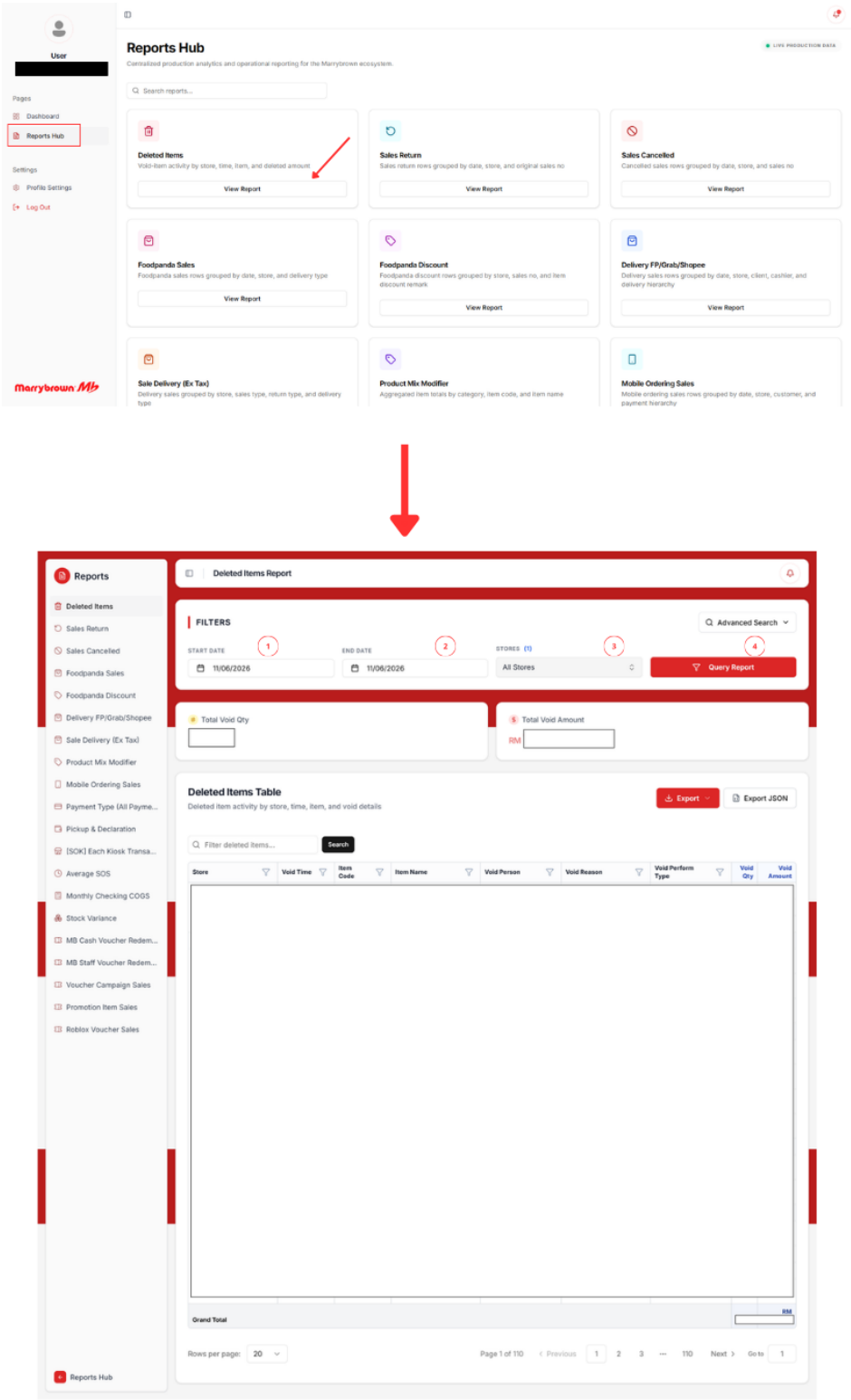

Figure 5.1: Reports Hub and representative report-query interface

The implemented portal also included role-restricted supporting features beyond ordinary report retrieval. Admin-only views were developed for data sync, automation control, and user management. These pages extended the platform beyond

passive report viewing and reflected the fact that the delivered system functioned as an internal reporting platform with controlled operational administration, not only as a collection of static report pages.

5.2.4 Administrative and Operational Functions

Several administrative and operational functions were implemented to make the platform sustainable for real use after report delivery. The first was admin-managed user control under the authentication model, where normal user access to report pages was separated from admin-only access to management functions. The second was the portal-triggered manual sync workflow, through which administrators could request sales-window or reference refreshes and review request history and progress. The third was automation management, where authorised administrators could review scheduled-task status and request enable, disable, or schedule-change actions for routine ETL automation.

An additional operational enhancement was the data-quality checking layer for portal-triggered replication. This feature allowed administrators to review source-to-replica quality outcomes for selected reporting windows, perform re-check actions, and trigger repair-oriented follow-up where required. This function was relevant academically because it showed that report continuity depended not only on endpoint implementation, but also on the trustworthiness of the replicated data used by those endpoints.

The administrative layer described above was implemented as three coordinated portal surfaces, each tied to a distinct governance responsibility. Figure 5.2 corresponds to user administration, where authorised staff review accounts and maintain role-controlled access. Figure 5.3 corresponds to data-sync administration, where replication requests, execution histories, and related follow-up actions are managed from the portal. Figure 5.4 corresponds to automation administration, where scheduled ETL activity is reviewed and change requests are handled under controlled

access. Considered together, these figures reinforce that the delivered platform supported governed internal operations in addition to ordinary report retrieval.

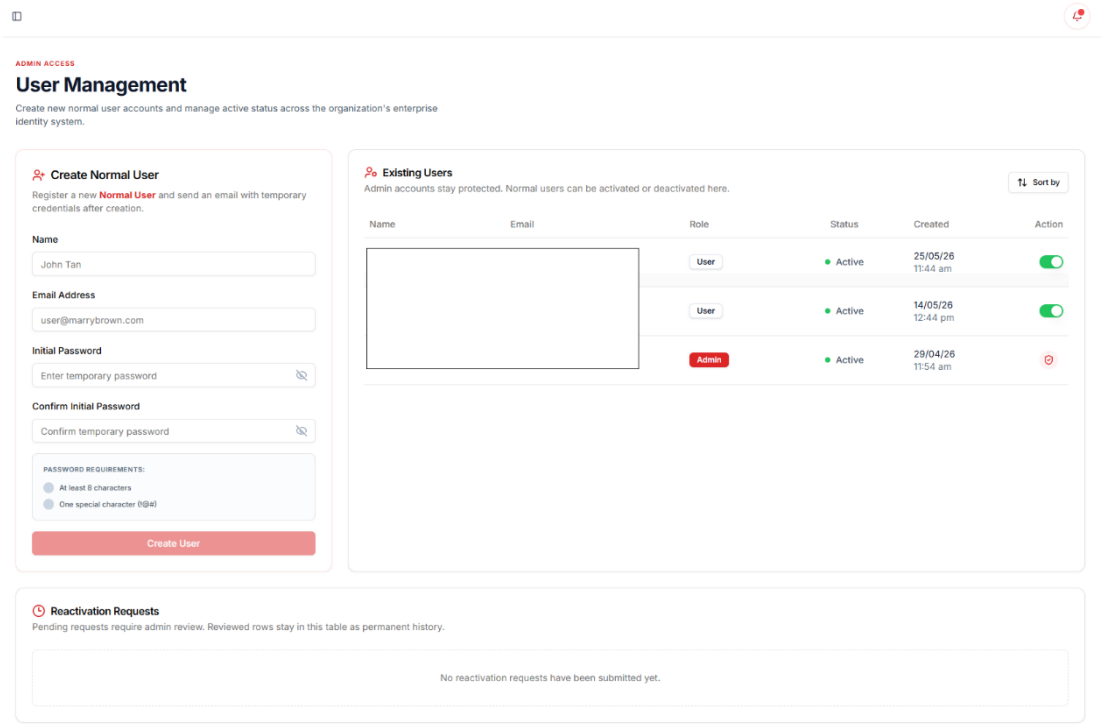

Figure 5.2: User Management page

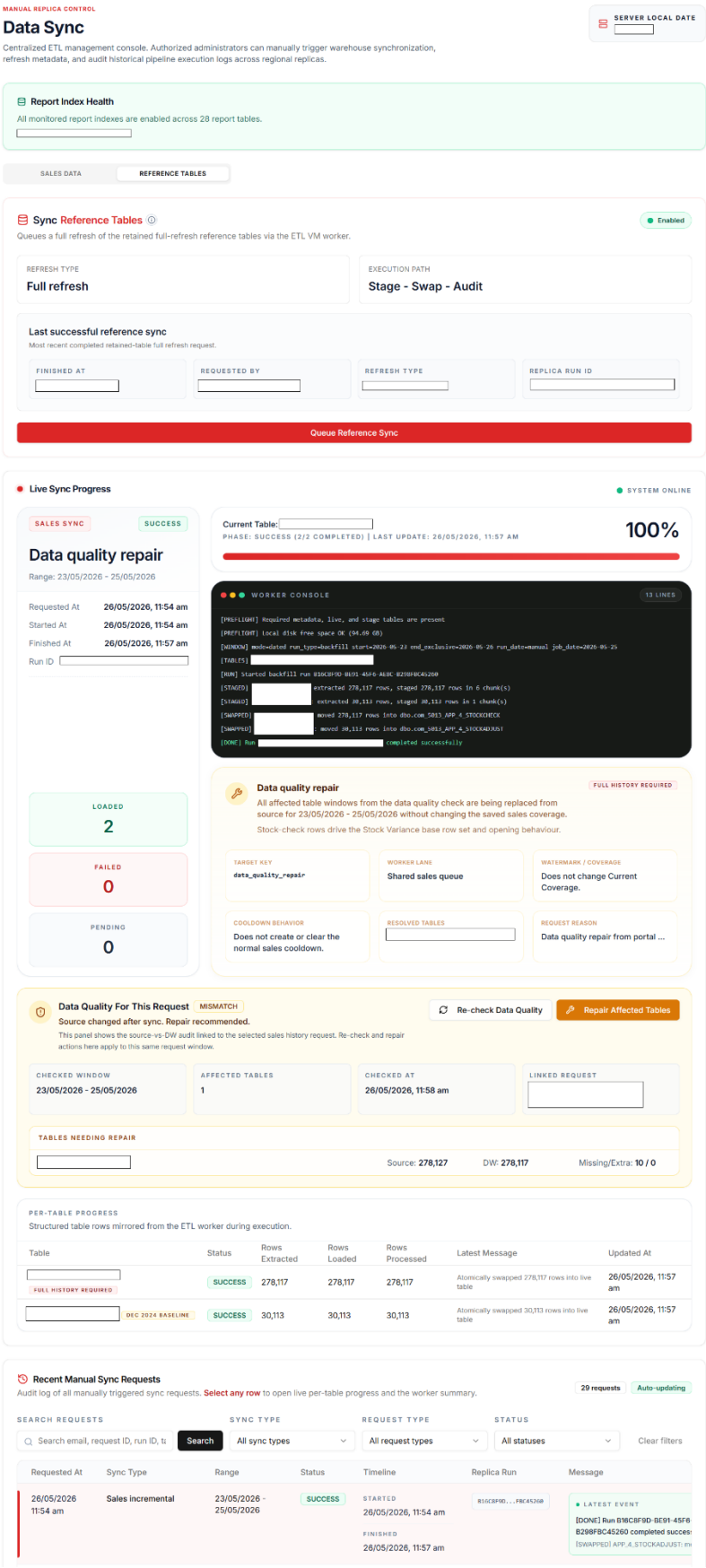

Figure 5.3: Data Sync page

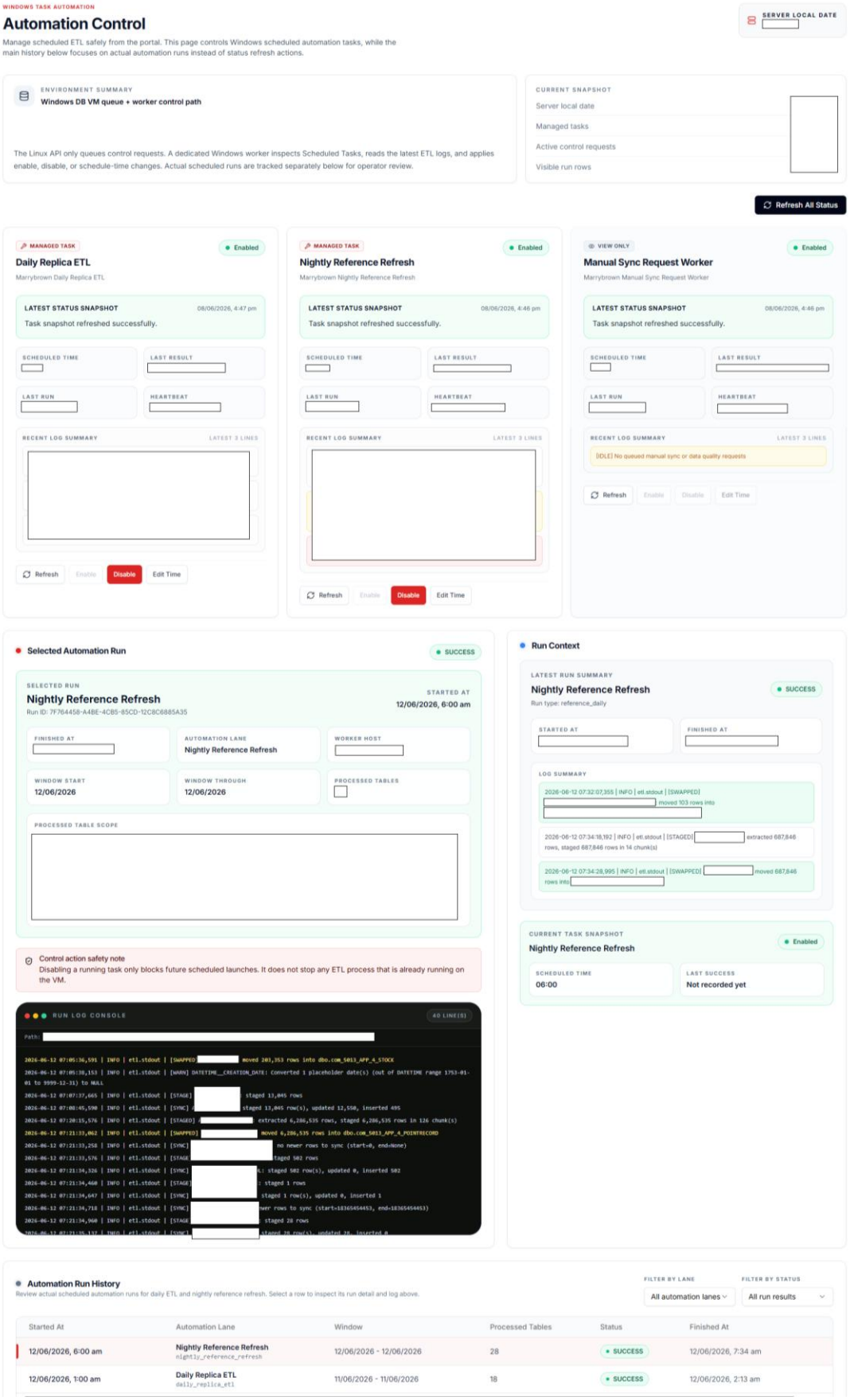

Figure 5.4: Automation Control page

5.2.5 Report Implementation Coverage

Within the formal implementation and business scope, the platform was developed to support 19 sales and payment reports. Of these, 16 reports were successfully implemented and retained in the delivered active reporting surface. Three reports were not fully closed within the project boundary and are documented as limitations rather than omitted outcomes. In addition to the formal 19-report scope, three additional customised reports were also developed based on company requests during implementation.

The retained formal-scope reports covered several reporting groups, including sales and exception control, payment and voucher reconciliation, delivery and ordering channels, and product-, stock-, or cost-related operational analysis. The additional customised reports extended the implemented platform beyond the original formal scope and demonstrated that the shared ETL, backend, and portal architecture could support company-requested report families without requiring a separate system. Detailed per-report specifications, statuses, and classifications are documented in Appendix A.

The three formal-scope reports that were not fully closed were Product Mix Report, Discount Remark Report, and Product Mix with modifier without ETL. These were not left unresolved due to a simple lack of page development. Rather, they remained unresolved after reverse engineering and parity validation because the remaining logic required for full closure, especially profit-related behaviour in the product-mix area, could not be isolated conclusively within the available project boundary.

5.3 Coding of the system's main functions/Process

5.3.1 Data Replication and Refresh Process

The main data-process function of the platform was the movement of selected source data into the reporting database in a form suitable for report reconstruction. This process followed a staged boundary-based refresh pattern. For dated-table maintenance, the ETL process identified a defined reporting window, extracted source rows for that window, staged the refreshed data, replaced the corresponding live rows in the reporting database, and then recorded run metadata and progress information. This approach ensured that rerunning the same reporting window remained deterministic and that failed or incomplete windows could be retried without depending on ad hoc manual cleanup.

This process was important because reporting mismatches were not caused only by query logic. In several cases, mismatches were associated with late source edits, historical incompleteness, or replica-state inconsistencies. The implemented refresh model therefore had to support not only normal data movement but also controlled replays and focused correction work. For broad historical loading, manual utilities were used for baseline and backfill operations. For routine maintenance, scheduled daily ETL and portal-triggered sync requests used the staged orchestrator path. This split between baseline loading and operational refresh was necessary to balance reliability, recoverability, and maintainability.

The implemented process also included quality-oriented follow-up for selected portal-triggered sync windows. After a successful sales-window refresh, a source-to-replica quality check could be launched for the same window. Where mismatches were identified, re-check or repair-oriented follow-up could be requested. This meant that the data process for the platform was not limited to extraction and loading alone; it also included mechanisms for checking whether the reporting replica remained trustworthy enough for subsequent report reconstruction.

This two-path refresh model is summarised in Figures 5.5 and 5.6. Figure 5.5 represents the retained reference-table refresh path, in which a full refresh request is initiated from the portal and then monitored through staged execution and per-table progress review. Figure 5.6 represents the dated sales-data refresh path, where a bounded sync request moves through queued execution, post-run data-quality review, and repair-oriented follow-up for the selected reporting window. Presented in this order, the figures support the subsection's argument that portal-triggered replication was implemented as a controlled and auditable administrative process rather than as an opaque background task.

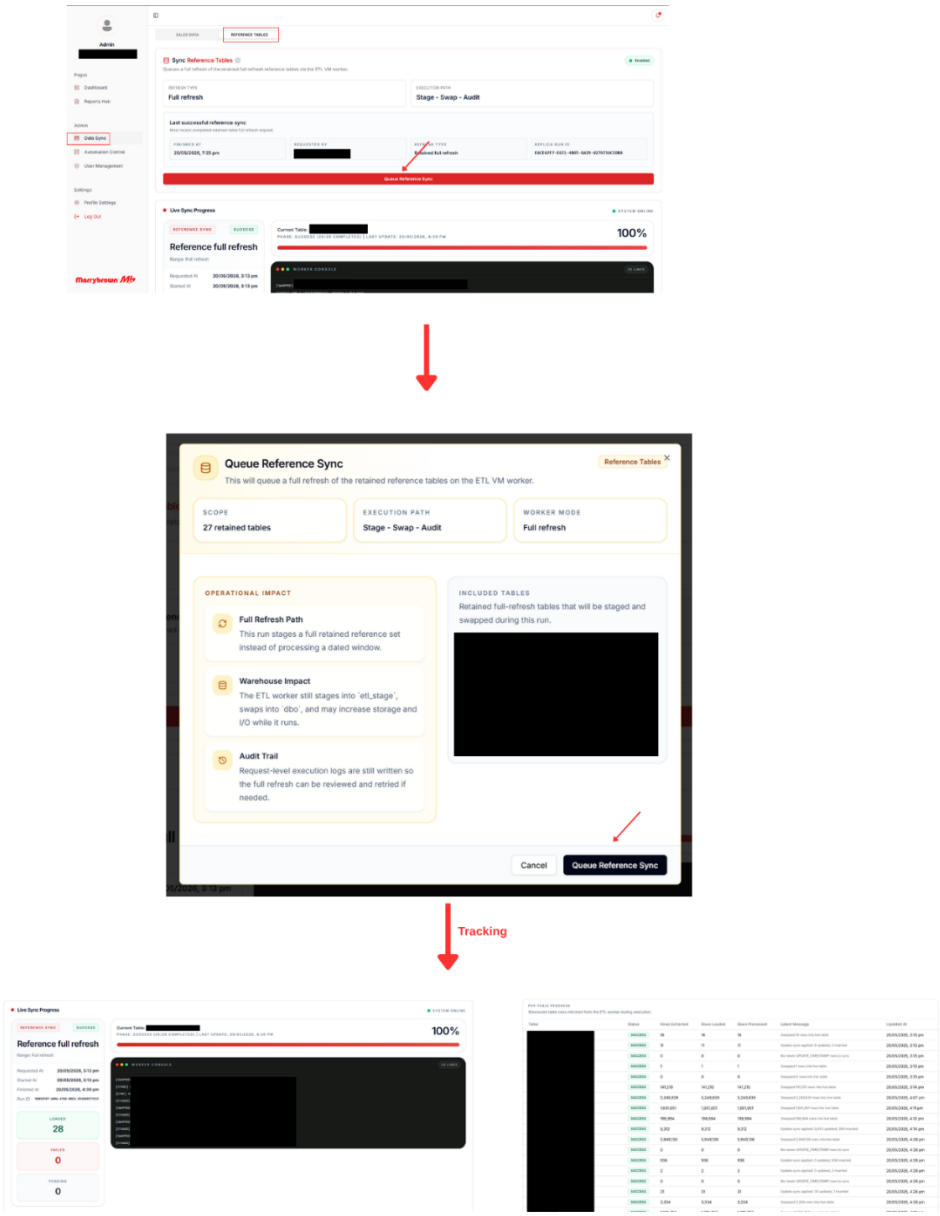

Figure 5.5: Portal-triggered reference-table refresh and execution tracking

Figure 5.6: Portal-triggered dated sales-data replication and data-quality review

5.3.2 Report Reconstruction Process

The main report-building process began with identifying the report behaviour expected from the vendor portal, including required parameters, visible output fields, grouping behaviour, totals, and export structure. From that baseline, the contributing datasets were traced in the replicated database and the required relationships were reconstructed at query level. This process was repeated across reports, even though the exact grouping and metric logic differed from one report family to another.

Once the contributing data paths were understood, the report logic was implemented in the FastAPI semantic/API layer. This typically included parameter validation, date-window scoping, outlet filtering, sales-status or payment-status handling where relevant, grouping and aggregation rules, and output formatting suitable for frontend rendering and export. The portal did not simply render raw SQL output. Instead, the backend response contracts were shaped to support tabular display, grouped subtotal behaviour, advanced search, and export actions in a stable and repeatable way.

Although each report had unique business rules, the implemented process remained consistent at high level. All report modules followed the same core pattern of source tracing, semantic reconstruction, parameter-driven querying, structured output shaping, and parity-oriented validation. This shared process allowed the project to scale from the retained formal report set to additional customised reports without abandoning the same platform architecture.

This reconstruction pattern is summarised in Figure 5.7. The figure represents the general backend process through which a report request moves from user parameter selection to semantic validation, replicated-data retrieval, dataset joining, grouping and derived-field handling, and final response shaping for portal display and export. Although the detailed query logic differs across report modules, the same high-level reconstruction sequence was retained throughout the implementation. This consistency was important because it allowed the platform to support multiple formal-scope and

customised reports within the same semantic/API architecture while preserving traceability for parity validation.

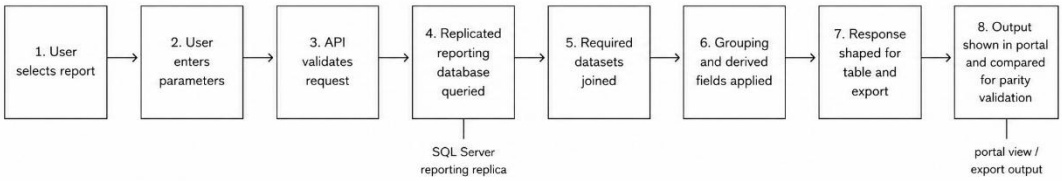

Figure 5.7: Sanitised illustration of report reconstruction logic

5.3.3 Payment Type Report Development

The Payment Type (All Payment) report is a suitable formal-scope example because it was operationally important, highly sensitive to row-level grouping behaviour, and representative of the reverse-engineering work required by the project. The purpose of this report was to provide a detailed breakdown of payment collections by store, order source, payment method, device, card type, and reference. The challenge was not merely to total payment rows, but to reconstruct the same grouping and display behaviour expected from the vendor portal with exactness sufficient for reconciliation work.

Implementation began by identifying the required payment-, sales-, and location-level source relationships in the replicated database. The main data sources were payment rows, sales status and order-source fields, and location-name mappings. Parameter handling was then defined around the selected date range, optional outlet scope, sales-status filtering, and payment-status behaviour. Because the report was intended for parity-sensitive operational use, the backend response also had to preserve a stable flat-row structure that could later support frontend grouped rendering and export processing.

The more difficult part of the implementation lay in the reference-grouping behaviour. Payment references could appear in multiple case variants, while portal output would still present them under a single preferred display value. Additional care was also required for payment-method categorisation, exclusion of return-sales behaviour, handling of zero-time anomalies, and consistent inclusion of payment rows in the correct collection totals. These issues made the report representative of the wider project challenge: the output could not be matched reliably through naive grouping alone, and several rounds of reverse engineering and refinement were required before parity could be accepted.

5.3.4 Voucher Campaign & Reward Sales Report Development

The Voucher Campaign & Reward Sales Report is a suitable customised-report example because it demonstrates that the implemented platform supported not only vendor-aligned report reconstruction, but also additional company-requested analysis built on the same architecture. This report was developed as a dynamic report family for eligible voucher campaigns and points rewards, while keeping the earlier Roblox-specific report intact as a separate implemented report.

The implementation process began with investigation of how voucher campaign and reward data could be represented reliably using the replicated data model. Rather than anchoring the report on a fixed voucher name, the implemented design used a selectable redemption-type identity so that the report could be applied to multiple campaign or reward families. The backend then reconstructed sale-level voucher scope, basket rows, derived bundle grouping, and additional-purchase indicators. This required not only data retrieval, but also interpretation of basket structure so that the final output could distinguish between the target bundle itself, other vouchers in the same sale, and additional paid items.

On the portal side, the customised report reused the shared report architecture but introduced a selector-driven reporting flow before ordinary date and outlet filtering. This illustrates a key implementation result of the project: once the shared

ETL, semantic/API, and portal architecture was stable, it became feasible to add company-requested customised reports without changing the overall platform model. The report therefore serves as evidence that the delivered system functioned as an extensible internal reporting platform rather than only as a one-time replication exercise.

5.3.5 Query Performance Observation

During implementation, selected report modules also underwent query-shape refinement and warehouse-side tuning to improve practical response time without changing validated business output. This work was documented through before-and-after timing checks on representative report scopes and was supported by later portal-side timing observations under real usage-style parameter sets.

One of the clearest examples was the Payment Type (All Payment) report, where the implementation logs recorded both application-side query-path refinement and later warehouse-side index recovery.

Table 5.1 summarises the strongest recorded timing observations for the Payment Type (All Payment) report.

Tested scope	**Before optimisation**	**After optimisation**	**Observation**
Single-store, single-day, 4-row case	approximately 63s	approximately 3.3s	substantial reduction after backend query-path refinement
Single-store, single-day, 608-row case	timed out beyond 120s	approximately 8.9s	timeout path removed after backend query-path refinement

Busy single-store, one-day sample after index recovery	not restated in the later log pass	approximately 1.1s	strong practical result after rebuilding disabled sales indexes
All-store, one-day sample after index recovery	not restated in the later log pass	approximately 12.2s	acceptable all-store one-day timing after warehouse-side tuning

Table 5.1: Recorded timing observations for Payment Type (All Payment)

The Payment Type observations indicate that performance improvement was achieved not only through report-query restructuring, but also through supporting warehouse maintenance and index recovery work.

Additional implementation logs recorded similar timing checks for other report families. Table 5.2 presents a compact summary of selected supporting observations.

Report	**Tested scope**	**Recorded timing change or practical timing**	**Interpretation**
Mobile Ordering Sales	All stores, 2025-01-01 to 2025-01-05	approximately 4.83s to 4.38s	modest improvement after query-shape refinement
Delivery-FoodPanda,Grabfood,ShopeeFood	All stores, 2025-01-01 to 2025-01-05	approximately 29.63s to 27.39s; later portal-side timing almost 20s	moderate backend improvement with usable portal-side outcome

Foodpanda Discount	All stores, 2025-01-01 to 2025-01-05	approximately 37.91s to 25.90s; later portal-side timing around 40+s	broad-window improvement, though still relatively heavy in portal use
Sales Return Report	All stores, 2025-01-01 to 2025-01-05	approximately 5.53s to 6.04s; later portal-side timing around 10s	timing was recorded and operationally acceptable, but not a strong improvement case
Sales Cancelled Report	All stores, 2025-01-01 to 2025-01-05	approximately 9.86s to 10.56s; later portal-side timing almost 20+s	timing was recorded and operationally acceptable, but not a strong improvement case
Deleted Items	All stores, 2025-01-01 to 2025-01-05	approximately 8.21s to 7.41s; later portal-side timing almost 8s	small but measurable improvement with acceptable practical timing

Table 5.2: Selected supporting timing observations across optimised report modules

Taken together, the recorded outcomes show that some report paths improved materially, while others improved more modestly or remained relatively heavy for broad all-store reporting windows. Practical portal-side timing checks were also

recorded after optimisation to ensure that the observed improvements remained meaningful from the user perspective rather than only at backend execution level.

These observations should be interpreted as implementation evidence of operational tuning rather than as a formal benchmark against the vendor portal. Within the project boundary, the main purpose of this work was to reduce timeout risk, improve response consistency, and keep validated report modules usable in the internal reporting workflow.

5.3.6 Report Accuracy Validation

The strongest testing evidence for the project was the report-accuracy validation process. This process was distinct from UAT because its purpose was not merely to confirm that a page loaded or exported correctly, but to confirm that the rebuilt report output was sufficiently aligned with the vendor portal under the same report conditions. The general validation approach was to run the platform report and the vendor portal report using the same date range, outlet scope, and other relevant filters, then compare row-level output, grouping behaviour, totals, subtotals, and exported results.

For the Payment Type (All Payment) report, this validation process was especially important because the report was sensitive to grouping precision at payment-reference level. The validation work therefore involved repeated comparison between platform output and vendor export output, investigation of mismatched rows or totals, and refinement of the reconstructed query logic until acceptable parity was achieved. Where anomalies remained, the investigation distinguished between logic faults and data-quality issues in the replica state, so that the report logic itself was not altered incorrectly to compensate for replica-specific data problems.

The same validation pattern was applied across the wider retained report surface. For each report module, parity validation used traceable filter conditions and visible output comparison rather than unsupported assumptions about hidden vendor

logic. This validation discipline was a defining characteristic of the project because it anchored report acceptance to evidence rather than to superficial interface completion. The detailed validation evidence is represented by the PV test cases shown in Figures 5.8 to 5.15.

Figure 5.8 shows the comparison-based validation used to check payment grouping, totals, and export consistency for the Payment Type report.

PV01: Payment Type (All Payment) Report Accuracy Validation

Report accuracy validation evidence. This layout is comparison-based and is intentionally different from the UAT test case template.

Report / Process	Payment Type (All Payment)	**Validation Type**	Report output reconciliation
Test Data Scope	Selected outlet and date range	**Validation Source**	Platform output compared with vendor portal output
Reviewer	Project developer and business user	**Execution Date**	2026-05-20

Validation Focus:
Validates payment-method grouping, outlet/date filters, transaction counts, totals, and export output.

Comparison Setup

Platform Output Evidence	Vendor / Reference Output Evidence	Filter Alignment	Comparison Level	Validation Method	Reviewer Note	Result	Evidence Location	Sensitivity Check	Public-Safe Note
Portal/API generated output and export file	Vendor portal output or approved business comparison output	Same selected outlet/date/report criteria	Row, group, subtotal, total, and export level	Manual reconciliation supported by sampled row checks	Reviewed by developer and business user	**Passed**	Screenshot/export kept separately for final report evidence	Credentials and internal access paths excluded	Only report behaviour and validation outcome documented

Reconciliation Checklist

Comparison Point	Expected Match / Rule	Platform Observation	Vendor / Reference Observation	Result	Notes
Filter criteria	Same outlet and date range are applied	Platform accepted identical report parameters	Vendor portal export was generated using the same criteria	**Matched**	No filter mismatch identified
Payment grouping	Payment method categories are grouped consistently	Payment groups displayed under expected headings	Portal output used the same payment grouping pattern	**Matched**	Grouping names were checked manually
Transaction count	Number of payment records should match for the selected range	Record count aligned after refreshed replica data was used	Portal export showed the same transaction scope	**Matched**	Validated after sync completion
Total amount	Report-level total should reconcile	Displayed total matched the comparison value	Portal total matched the same total amount	**Matched**	Rounding reviewed at two-decimal level
Export output	Exported file should preserve displayed columns	Excel export contained the same visible report fields	Portal export was used as comparison evidence	**Matched**	Column order checked for report readability

Variance / Exception Handling

Data Item / Field	Variance Type	Investigation / Handling Action	Resolution Status
None identified	N/A	No adjustment required after comparison	Closed

Validation Conclusion

The reconstructed output was accepted for the validated scenario because the selected rows, grouping, totals, and export behaviour reconciled with the comparison evidence.

Figure 5.8: PV01 Payment Type (All Payment) report accuracy validation

Figure 5.9 records the validation of return retrieval, negative-value handling, and related report totals for the Sales Return Report.

PV02: Sales Return Report Accuracy Validation

Report accuracy validation evidence. This layout is comparison-based and is intentionally different from the UAT test case template.

Report / Process	Sales Return	**Validation Type**	Report output reconciliation
Test Data Scope	Return transactions within selected date range	**Validation Source**	Platform output compared with vendor portal output
Reviewer	Project developer and business user	**Execution Date**	2026-05-21

Validation Focus:
Validates return transaction retrieval, negative amount handling, return item details, and outlet/date filtering.

Comparison Setup

Platform Output Evidence	Vendor / Reference Output Evidence	Filter Alignment	Comparison Level	Validation Method	Reviewer Note	Result	Evidence Location	Sensitivity Check	Public-Safe Note
Portal/API generated output and export file	Vendor portal output or approved business comparison output	Same selected outlet/date/report criteria	Row, group, subtotal, total, and export level	Manual reconciliation supported by sampled row checks	Reviewed by developer and business user	**Passed**	Screenshot/export kept separately for final report evidence	Credentials and internal access paths excluded	Only report behaviour and validation outcome documented

Reconciliation Checklist

Comparison Point	Expected Match / Rule	Platform Observation	Vendor / Reference Observation	Result	Notes
Date and outlet filters	Return report should only show selected business period and outlet	Returned rows followed selected parameter values	Portal output used equivalent period and outlet	**Matched**	Filter controls checked before comparison
Returned item details	Item lines should represent returned products only	Return item rows were displayed with expected quantities	Portal export contained the same return-line scope	**Matched**	Duplicate checking performed
Amount sign treatment	Return amounts should be represented consistently	Platform applied the expected return amount presentation	Portal output used the same direction of amount treatment	**Matched**	No manual sign conversion required
Subtotal and report total	Computed total should reconcile with portal result	Subtotal and grand total aligned	Portal total matched the same comparison amount	**Matched**	Rounding reviewed
Export result	Export should preserve selected report output	Export file generated successfully	Portal comparison export was available	**Matched**	No field loss observed

Variance / Exception Handling

Data Item / Field	Variance Type	Investigation / Handling Action	Resolution Status
None identified	N/A	No adjustment required after comparison	Closed

Validation Conclusion

The reconstructed output was accepted for the validated scenario because the selected rows, grouping, totals, and export behaviour reconciled with the comparison evidence.

Figure 5.9: PV02 Sales Return Report accuracy validation

Figure 5.10 presents the validation of cancellation-related rows, output fields, and export behaviour for the Sales Cancelled Report.

PV03: Sales Cancelled Report Accuracy Validation

Report accuracy validation evidence. This layout is comparison-based and is intentionally different from the UAT test case template.

Report / Process	Sales Cancelled	Validation Type	Report output reconciliation
Test Data Scope	Cancelled transactions in selected business period	Validation Source	Platform output compared with vendor portal output
Reviewer	Project developer and business user	Execution Date	2026-05-21

Validation Focus:
Validates cancelled order inclusion rules, cancellation-related columns, totals, and export behaviour.

Comparison Setup

Platform Output Evidence	Vendor / Reference Output Evidence	Filter Alignment	Comparison Level	Validation Method	Reviewer Note	Result	Evidence Location	Sensitivity Check	Public-Safe Note
Portal/API generated output and export file	Vendor portal output or approved business comparison output	Same selected outlet/date/report criteria	Row, group, subtotal, total, and export level	Manual reconciliation supported by sampled row checks	Reviewed by developer and business user	Passed	Screenshot/export kept separately for final report evidence	Credentials and internal access paths excluded	Only report behaviour and validation outcome documented

Reconciliation Checklist

Comparison Point	Expected Match / Rule	Platform Observation	Vendor / Reference Observation	Result	Notes
Cancellation scope	Only cancelled sales should appear	Rows represented cancelled transactions only	Portal comparison output used the same cancelled-sales scope	Matched	Non-cancelled transactions excluded
Order identifiers	Cancelled transaction references should align	Displayed transaction identifiers matched sampled portal records	Portal export showed same sampled identifiers	Matched	Sampling performed across outlets
Cancellation details	Reason/status fields should remain traceable where available	Cancellation-related fields displayed in expected columns	Portal output contained equivalent cancellation references	Matched	Missing optional values treated as blank
Total amount	Cancelled sales amount should reconcile	Displayed total matched portal comparison total	Portal output total used for reconciliation	Matched	Rounding checked
Export result	Export should preserve cancelled sales rows and columns	Export generated and retained visible fields	Portal output used as comparison record	Matched	Export column visibility checked

Variance / Exception Handling

Data Item / Field	Variance Type	Investigation / Handling Action	Resolution Status
None identified	N/A	No adjustment required after comparison	Closed

Validation Conclusion

The reconstructed output was accepted for the validated scenario because the selected rows, grouping, totals, and export behaviour reconciled with the comparison evidence.

Figure 5.10: PV03 Sales Cancelled Report accuracy validation

Figure 5.11 shows the reconciliation used to verify delivery sales calculation excluding tax under aligned outlet and date filters.

PV04: Sale Delivery Ex Tax Report Accuracy Validation

Report accuracy validation evidence. This layout is comparison-based and is intentionally different from the UAT test case template.

Report / Process	Sale Delivery Ex Tax	Validation Type	Report output reconciliation
Test Data Scope	Delivery transactions for selected outlet/date range	Validation Source	Platform output compared with vendor portal output
Reviewer	Project developer and business user	Execution Date	2026-05-22

Validation Focus:
Validates delivery sales calculation excluding tax, delivery-channel grouping, and date/outlet filtering.

Comparison Setup

Platform Output Evidence	Vendor / Reference Output Evidence	Filter Alignment	Comparison Level	Validation Method	Reviewer Note	Result	Evidence Location	Sensitivity Check	Public-Safe Note
Portal/API generated output and export file	Vendor portal output or approved business comparison output	Same selected outlet/date/report criteria	Row, group, subtotal, total, and export level	Manual reconciliation supported by sampled row checks	Reviewed by developer and business user	Passed	Screenshot/export kept separately for final report evidence	Credentials and internal access paths excluded	Only report behaviour and validation outcome documented

Reconciliation Checklist

Comparison Point	Expected Match / Rule	Platform Observation	Vendor / Reference Observation	Result	Notes
Delivery transaction scope	Only delivery-related transactions should be included	Displayed rows followed delivery scope	Portal output used same delivery report context	Matched	Channel filtering reviewed
Ex-tax amount calculation	Sales amount should exclude tax component	Platform amount matched ex-tax comparison value	Portal result used as ex-tax reference	Matched	Calculation checked at sampled row and total level
Grouping by outlet/date	Rows should group consistently by selected outlet and date	Output grouped using expected outlet/date fields	Portal export followed equivalent grouping	Matched	Group headers reviewed
Report total	Final total should reconcile	Displayed total matched comparison total	Portal export showed same report total	Matched	No unexplained difference after data refresh
Export result	Export should preserve ex-tax values	Excel export preserved calculation columns	Portal export used as comparison evidence	Matched	No formatting issue affecting totals

Variance / Exception Handling

Data Item / Field	Variance Type	Investigation / Handling Action	Resolution Status
None identified	N/A	No adjustment required after comparison	Closed

Validation Conclusion

The reconstructed output was accepted for the validated scenario because the selected rows, grouping, totals, and export behaviour reconciled with the comparison evidence.

Figure 5.11: PV04 Sale Delivery Ex Tax report accuracy validation

Figure 5.12 records the validation of voucher redemption rows, grouped totals, and period-based filtering for the MB Cash Voucher report.

PV05: MB Cash Voucher Report Accuracy Validation

Report accuracy validation evidence. This layout is comparison-based and is intentionally different from the UAT test case template.

Report / Process	MB Cash Voucher	**Validation Type**	Report output reconciliation
Test Data Scope	Voucher redemption activity in selected period	**Validation Source**	Platform output compared with vendor portal output
Reviewer	Project developer and business user	**Execution Date**	2026-05-22

Validation Focus:
Validates voucher redemption rows, voucher amounts, outlet/date filters, and grouped totals.

Comparison Setup

Platform Output Evidence	Vendor / Reference Output Evidence	Filter Alignment	Comparison Level	Validation Method	Reviewer Note	Result	Evidence Location	Sensitivity Check	Public-Safe Note
Portal/API generated output and export file	Vendor portal output or approved business comparison output	Same selected outlet/date/report criteria	Row, group, subtotal, total, and export level	Manual reconciliation supported by sampled row checks	Reviewed by developer and business user	**Passed**	Screenshot/export kept separately for final report evidence	Credentials and internal access paths excluded	Only report behaviour and validation outcome documented

Reconciliation Checklist

Comparison Point	Expected Match / Rule	Platform Observation	Vendor / Reference Observation	Result	Notes
Voucher redemption scope	Only relevant voucher redemption rows should appear	Voucher rows were filtered to the selected redemption scope	Portal export reflected the same voucher activity	**Matched**	Non-voucher payment rows excluded
Voucher amount	Voucher amount should match row-level comparison	Sampled voucher amounts aligned	Portal sampled rows showed same amounts	**Matched**	Sampled across more than one outlet
Outlet/date grouping	Report grouping should follow selected outlet/date parameters	Output followed expected grouping	Portal output grouped in the same business context	**Matched**	Group totals checked
Grand total	Total voucher value should reconcile	Platform total aligned with comparison total	Portal output showed same total value	**Matched**	Rounding reviewed
Export result	Export should preserve voucher identifiers and values	Export file generated with visible voucher fields	Portal export used for comparison	**Matched**	No field omission observed

Variance / Exception Handling

Data Item / Field	Variance Type	Investigation / Handling Action	Resolution Status
None identified	N/A	No adjustment required after comparison	Closed

Validation Conclusion

The reconstructed output was accepted for the validated scenario because the selected rows, grouping, totals, and export behaviour reconciled with the comparison evidence.

Figure 5.12: PV05 MB Cash Voucher redemption accuracy validation

Figure 5.13 shows the validation of customised campaign filtering, reward grouping, and campaign-level totals for the Voucher Campaign & Reward Sales report.

PV06: Voucher Campaign & Reward Sales Report Accuracy Validation

Report accuracy validation evidence. This layout is comparison-based and is intentionally different from the UAT test case template.

Report / Process	Voucher Campaign & Reward Sales	**Validation Type**	Custom report validation
Test Data Scope	Selected campaign period and reward/voucher criteria	**Validation Source**	Platform output compared with business-requested rules and portal evidence
Reviewer	Project developer and business user	**Execution Date**	2026-05-24

Validation Focus:
Validates customised voucher campaign filtering, reward sales grouping, and campaign-level totals.

Comparison Setup

Platform Output Evidence	Vendor / Reference Output Evidence	Filter Alignment	Comparison Level	Validation Method	Reviewer Note	Result	Evidence Location	Sensitivity Check	Public-Safe Note
Portal/API generated output and export file	Vendor portal output or approved business comparison output	Same selected outlet/date/report criteria	Row, group, subtotal, total, and export level	Manual reconciliation supported by sampled row checks	Reviewed by developer and business user	**Passed**	Screenshot/export kept separately for final report evidence	Credentials and internal access paths excluded	Only report behaviour and validation outcome documented

Reconciliation Checklist

Comparison Point	Expected Match / Rule	Platform Observation	Vendor / Reference Observation	Result	Notes
Campaign filter	Only requested campaign/reward rows should be included	Platform filtered rows using the configured campaign criteria	Comparison evidence confirmed expected campaign scope	**Matched**	Custom request reviewed with business user
Voucher/reward identification	Voucher and reward records should be classified correctly	Rows were separated into the expected campaign/reward categories	Reference output supported same categorisation	**Matched**	Category samples checked
Outlet and date filters	Selected outlet/date criteria should limit the output	Output respected selected criteria	Reference output used same period and outlet context	**Matched**	Parameter handling verified
Campaign total	Total campaign-related value should reconcile	Campaign total matched validation total	Comparison evidence matched the same total	**Matched**	Rounding reviewed
Export result	Export should support operational review	Export contained campaign fields and values	Reference export used for validation	**Matched**	No sensitive setup data included

Variance / Exception Handling

Data Item / Field	Variance Type	Investigation / Handling Action	Resolution Status
None identified	N/A	No adjustment required after comparison	Closed

Validation Conclusion

The reconstructed output was accepted for the validated scenario because the selected rows, grouping, totals, and export behaviour reconciled with the comparison evidence.

Figure 5.13: PV06 Voucher Campaign and Reward Sales accuracy validation

Figure 5.14 presents the validation evidence for portal-triggered data-quality checking after a manual sync window.

PV07: Portal-Triggered Data Quality Validation

Report accuracy validation evidence. This layout is comparison-based and is intentionally different from the UAT test case template.

Report / Process	Portal-triggered replication quality check	Validation Type	Data completeness and consistency validation
Test Data Scope	Selected reporting period after manual sync	Validation Source	Replica status, quality-check result, and report output comparison
Reviewer	Project developer and business user	Execution Date	2026-05-25

Validation Focus:
Validates that replicated reporting data can be checked and rechecked after a portal-triggered sync request.

Comparison Setup

Platform Output Evidence	Vendor / Reference Output Evidence	Filter Alignment	Comparison Level	Validation Method	Reviewer Note	Result	Evidence Location	Sensitivity Check	Public-Safe Note
Portal/API generated output and export file	Vendor portal output or approved business comparison output	Same selected outlet/date/report criteria	Row, group, subtotal, total, and export level	Manual reconciliation supported by sampled row checks	Reviewed by developer and business user	Passed	Screenshot/export kept separately for final report evidence	Credentials and internal access paths excluded	Only report behaviour and validation outcome documented

Reconciliation Checklist

Comparison Point	Expected Match / Rule	Platform Observation	Vendor / Reference Observation	Result	Notes
Sync request completion	Manual sync should finish before validation is performed	Sync status reached completed state	Operational status view confirmed completion	Matched	No direct server credentials documented
Row-count reasonableness	Replica row counts should be consistent for selected period	Quality check returned expected completeness result	Operational comparison did not show missing period after refresh	Matched	Focused on reporting readiness
Missing-period detection	System should identify incomplete data when present	Quality check exposed incomplete state before repair/recheck where applicable	Expected behaviour observed	Matched	Scenario tested using controlled period selection
Recheck after repair/sync	Recheck should update status after data refresh	Recheck result changed to valid after refreshed data was available	Operational status reflected updated state	Matched	Closed through supported portal path
Report readiness	Reports should be generated only against refreshed replica data	Report validation used refreshed replica period	Comparison output used equivalent business period	Matched	Supports final report accuracy validation

Variance / Exception Handling

Data Item / Field	Variance Type	Investigation / Handling Action	Resolution Status
Incomplete historical periods	Operational data completeness issue	Handled through manual sync and quality recheck	Closed for validated periods

Validation Conclusion

The reconstructed output was accepted for the validated scenario because the selected rows, grouping, totals, and export behaviour reconciled with the comparison evidence.

Figure 5.14: PV07 Portal-triggered data quality validation

Figure 5.15 documents the remaining profit-logic limitation that prevented final closure for Product Mix related reporting and Discount Remark reporting.

PV08: Product Mix and Discount Profit Logic Validation Limitation

Report accuracy validation evidence. This layout is comparison-based and is intentionally different from the UAT test case template.

Report / Process	Product Mix / Discount Remark profit-related output	Validation Type	Exception and limitation record
Test Data Scope	Selected report samples with profit-related fields	Validation Source	Platform reconstruction compared with vendor portal output
Reviewer	Project developer and business user	Execution Date	2026-05-26

Validation Focus:
Documents the unresolved profit-logic limitation for Product Mix related reporting and Discount Remark reporting.

Comparison Setup

Platform Output Evidence	Vendor / Reference Output Evidence	Filter Alignment	Comparison Level	Validation Method	Reviewer Note	Result	Evidence Location	Sensitivity Check	Public-Safe Note
Portal/API generated output and export file	Vendor portal output or approved business comparison output	Same selected outlet/date/report criteria	Row, group, subtotal, total, and export level	Manual reconciliation supported by sampled row checks	Reviewed by developer and business user	Not Fully Closed	Screenshot/export kept separately for final report evidence	Credentials and internal access paths excluded	Only report behaviour and validation outcome documented

Reconciliation Checklist

Comparison Point	Expected Match / Rule	Platform Observation	Vendor / Reference Observation	Result	Notes
Basic row scope	Product/discount rows should follow selected date and outlet filters	Row scope could be reconstructed for selected samples	Portal output provided comparable row scope	Matched	Base filtering was not the blocker
Quantity and sales values	Non-profit values should align where source logic was traceable	Quantity and sales-related fields were validated in sampled output	Portal output supported these comparisons	Matched	Validation focused on traceable fields
Profit column logic	Profit-related value should reconcile with portal result	Remaining profit calculation could not be isolated completely	Portal output did not provide enough observable evidence to derive final logic	Not Fully Closed	Documented as external black-box logic limitation
Report closure decision	Only fully validated reports should be included in retained active surface	Report remained documented but not treated as fully closed	Comparison limitation remained unresolved	Not Fully Closed	Avoided claiming false parity
Academic reporting treatment	Limitation should be stated honestly	Limitation recorded in final-report evidence	N/A	Documented	No internal credentials or proprietary query text included

Variance / Exception Handling

Data Item / Field	Variance Type	Investigation / Handling Action	Resolution Status
Profit column	Unresolved parity difference	Reverse engineering and validation were not sufficient to isolate final profit logic	Not fully closed

Validation Conclusion

The report area was documented as not fully closed because the remaining profit-related logic could not be isolated with sufficient confidence through reverse engineering and portal comparison evidence.

Figure 5.15: PV08 Product Mix and discount profit logic validation boundary

5.3.7 Black-Box Testing and User Acceptance Testing

Black-box testing and User Acceptance Testing (UAT) were treated together because, in the context of this project, both evaluated the system from the user-facing perspective rather than from the internal code path. These tests focused on whether users could sign in, access reports, enter parameters, retrieve outputs, inspect totals, export data, and use role-restricted administrative functions appropriately. The intent was to verify that the implemented platform supported realistic operational workflows rather than merely exposing technically correct endpoints.

The test scenarios were structured as test cases using the TC prefix, such as TC01, TC02, and subsequent cases. Core user-facing scenarios included authentication, report-catalogue access, report retrieval with valid filters, export behaviour, access to customised report modules, manual sync submission, data-quality follow-up visibility, automation-control access, and role-restricted administrative behaviour. This structure provided a clear and traceable record of functional acceptance across the implemented user workflows.

The UAT and black-box testing outcomes were intended to show that the implemented platform could be used coherently by both ordinary reporting users and authorised administrative users. This was especially relevant for the data-sync and automation functions, because these features extended the platform beyond simple report viewing and into controlled reporting operations. By structuring these tests around user-observable behaviour, the testing section complements the report-accuracy validation process rather than duplicating it.

The detailed UAT and black-box test cases are shown in Figures 5.16 to 5.31.

Figure 5.16 shows the functional acceptance check for valid user login and protected portal access.

User Login with Valid Credentials		Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
Description:	Verifies that an authorised user can log in and reach the protected portal area.	Priority:	High
		Test Type:	Functional / UAT

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC01	Verifies that an authorised user can log in and reach the protected portal area.	Registered active user account; login page accessible.	User email and password provided by test administrator.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Navigate to the login page.
2. Enter a valid email address.
3. Enter a valid password.
4. Click the login button.
5. Observe the landing page after authentication.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The user is redirected to the reporting portal after successful authentication.

Actual Outcome

The user logged in successfully and reached the protected portal page.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.16: TC01 user login with valid credentials

Figure 5.17 records the test confirming that an authenticated user can open the Reports Hub and review available report categories.

Reports Hub Access			
Description:	Verifies that an authenticated user can open the reports hub and view available report categories.	Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
		Priority:	High
		Test Type:	Functional / UAT

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC02	Verifies that an authenticated user can open the reports hub and view available report categories.	User is logged in; report definitions are available.	Authenticated user session.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Open the portal navigation menu.
2. Select Reports.
3. Review displayed report categories.
4. Select one report category.
5. Confirm that the report page opens.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The reports hub displays available report options and allows navigation to a report page.

Actual Outcome

Reports hub loaded successfully and report navigation worked as expected.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.17: TC02 Reports Hub access

Figure 5.18 presents the test used to confirm that common report parameters can be entered before report generation.

Report Parameter Entry			
Description:	Verifies that a user can enter common report filters such as date range and outlet before generating a report.	Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
		Priority:	High
		Test Type:	Functional / Black-box

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC03	Verifies that a user can enter common report filters such as date range and outlet before generating a report.	User has access to a report page.	Outlet/date range selected for testing.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Open a report page.
2. Select a valid start date.
3. Select a valid end date.
4. Select or confirm outlet/report filters.
5. Submit the report request.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The system accepts valid report parameters and starts report generation.

Actual Outcome

Parameters were accepted and the report request was processed.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.18: TC03 report parameter loading

Figure 5.19 shows the user-facing test for generating the Payment Type report through the portal workflow.

Generate Payment Type Report			
		Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
Description:	Verifies that a user can generate the Payment Type (All Payment) report from the portal.	Priority:	High
		Test Type:	Functional / UAT

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC04	Verifies that a user can generate the Payment Type (All Payment) report from the portal.	User is logged in and report data is available for selected period.	Payment Type report with selected outlet/date range.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Open Payment Type report.
2. Enter required filter values.
3. Click Search or Generate.
4. Wait for table output.
5. Review the loaded result rows.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The Payment Type report loads with grouped payment records and totals.

Actual Outcome

The report loaded with expected grouped rows and totals.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.19: TC04 Payment Type report query

Figure 5.20 records the review of grouped rows, subtotals, and grand totals after report generation.

Review Report Totals and Grouping			
		Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
Description:	Verifies that report rows, group totals, and grand totals are visible and understandable to the user.	Priority:	Medium
		Test Type:	Functional / Black-box

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC05	Verifies that report rows, group totals, and grand totals are visible and understandable to the user.	Report output has been generated.	Generated report result table.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Generate a report with valid filters.
2. Scroll through grouped report output.
3. Review subtotal rows where available.
4. Review grand total row.
5. Compare displayed values with expected validation evidence.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The report table presents grouped records and totals clearly.

Actual Outcome

Grouping and totals were displayed clearly for review.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.20: TC05 report totals review

Figure 5.21 shows the test confirming that generated report output can be exported for operational use.

Export Report Output			
Description:	Verifies that a user can export generated report data for operational use.	Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
		Priority:	High
		Test Type:	Functional / UAT

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC06	Verifies that a user can export generated report data for operational use.	Report output is already loaded on screen.	Generated report table.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Generate a report.
2. Click the export button.
3. Wait for file generation.
4. Open the exported file.
5. Compare exported columns with visible report output.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The exported file contains the displayed report fields and values.

Actual Outcome

Export completed successfully and contained the expected report output.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.21: TC06 report output export

Figure 5.22 presents the test for advanced search or table filtering over multi-row report output.

Advanced Search and Table Filtering			
Description:	Verifies that the user can narrow report output using available table search or filtering controls.	Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
		Priority:	Medium
		Test Type:	Functional / Black-box

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC07	Verifies that the user can narrow report output using available table search or filtering controls.	Report output contains multiple rows.	Search keyword or filter value.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Generate a report with multiple rows.
2. Enter a search keyword or filter value.
3. Apply the search/filter control.
4. Review filtered rows.
5. Clear the filter and confirm full result returns.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The table filters results according to the entered criteria and restores full output when cleared.

Actual Outcome

Filtering behaved as expected and the result table was restored after clearing.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.22: TC07 advanced search behaviour

Figure 5.23 records the user-facing retrieval test for the Sales Return Report under selected filters.

Generate Sales Return Report		Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
Description:	Verifies that a user can retrieve a sales-return related report from the portal.	Priority:	High
		Test Type:	Functional / UAT

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC08	Verifies that a user can retrieve a sales-return related report from the portal.	User is logged in and return records exist for the selected period.	Sales Return report filters.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Open Sales Return report.
2. Enter selected date and outlet filters.
3. Generate the report.
4. Review return transaction rows.
5. Review totals and export availability.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The Sales Return report displays return transaction rows for the selected filter.

Actual Outcome

Sales Return report loaded correctly with expected rows and totals.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.23: TC08 Sales Return Report query

Figure 5.24 shows the acceptance test for generating a customised company-requested report through the portal.

Generate Custom Voucher Campaign Report		Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
Description:	Verifies that a customised report requested by the company can be generated through the portal.	Priority:	High
		Test Type:	Functional / UAT

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC09	Verifies that a customised report requested by the company can be generated through the portal.	User is logged in; customised report is enabled.	Voucher campaign date/filter criteria.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Open the Voucher Campaign & Reward Sales report.
2. Enter campaign-related filter values.
3. Generate the report.
4. Review campaign rows and totals.
5. Export the result if required.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The customised report displays campaign-related sales/reward output according to selected criteria.

Actual Outcome

The customised report generated successfully and output was reviewable.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.24: TC09 customised report query

Figure 5.25 presents the access-control test confirming that normal users are restricted from admin-only pages.

Admin Page Access Restriction			
Description:	Verifies that administrative pages are restricted from normal report users.	Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
		Priority:	High
		Test Type:	Security / Black-box

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC10	Verifies that administrative pages are restricted from normal report users.	One normal user and one admin user are available for testing.	Normal user session and admin user session.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Log in as a normal user.
2. Attempt to access an admin-only page.
3. Observe access behaviour.
4. Log out and log in as admin user.
5. Open the same admin page.
6.
7.
8.

Postconditions: Access remains controlled based on role.

Execution Date: 2026-05-28

Expected Outcome: Normal user is blocked from admin functions, while admin user can access them.

Actual Outcome: Normal user could not access admin-only page and admin user could access it.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.25: TC10 normal-user admin restriction

Figure 5.26 records the test showing that an authorised admin user can open the Data Sync page and review sync controls.

Open Data Sync Administration Page			
Description:	Verifies that an admin user can open the data sync page and view sync-related controls/status.	Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
		Priority:	High
		Test Type:	Functional / UAT

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC11	Verifies that an admin user can open the data sync page and view sync-related controls/status.	Admin user is logged in.	Admin account and sync page route.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Log in as an admin user.
2. Open the Data Sync page.
3. Review available sync controls.
4. Review latest status area.
5. Confirm no unauthorised credentials are displayed.
6.
7.
8.

Postconditions: System state remains available for continued testing or user work.

Execution Date: 2026-05-28

Expected Outcome: The data sync page opens for admin users and shows appropriate controls/status information.

Actual Outcome: Data Sync page opened and displayed operational controls/status safely.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.26: TC11 admin Data Sync access

Figure 5.27 shows the workflow test for submitting a portal-triggered manual sync request.

Submit Portal-Triggered Manual Sync			
		Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
Description:	Verifies that an admin user can submit a manual sync request through the portal.	Priority:	High
		Test Type:	Functional / UAT

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC12	Verifies that an admin user can submit a manual sync request through the portal.	Admin user is logged in; sync service is available.	Selected sync period or sync option.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Open Data Sync page.
2. Select the required sync option.
3. Submit manual sync request.
4. Confirm the request prompt if displayed.
5. Review initial sync status.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The system accepts the sync request and records it for processing.

Actual Outcome

Manual sync request was submitted and appeared in the sync status view.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.27: TC12 manual sync request submission

Figure 5.28 presents the test used to monitor sync progress after a manual request has been submitted.

Monitor Sync Progress			
		Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
Description:	Verifies that an admin user can monitor portal-triggered sync progress after submission.	Priority:	Medium
		Test Type:	Functional / UAT

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC13	Verifies that an admin user can monitor portal-triggered sync progress after submission.	A manual sync request has been submitted.	Existing sync request record.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Open the sync status section.
2. Refresh or wait for status update.
3. Review current status.
4. Confirm completion or failure message.
5. Open related report after sync if needed.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The system displays an understandable sync status and completion result.

Actual Outcome

Sync progress/status was visible and understandable to the admin user.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.28: TC13 sync progress review

Figure 5.29 records the test for triggering or reviewing a post-sync data-quality re-check.

Run Data Quality Recheck			
	Author:	Project Developer	
	Reviewer:	Project Supervisor / Company User	
Description: Verifies that an admin user can trigger or review a data quality recheck after sync.	Priority:	High	
	Test Type:	Functional / UAT	

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC14	Verifies that an admin user can trigger or review a data quality recheck after sync.	Admin user is logged in; sync has completed.	Selected reporting period for quality check.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Open Data Sync or quality-check area.
2. Select the reporting period.
3. Run or refresh the quality check.
4. Review completeness/status result.
5. Use result to decide whether reports are ready.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The system displays data quality status for the selected period.

Actual Outcome

Data quality recheck completed and showed an understandable status result.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.29: TC14 data-quality re-check workflow

Figure 5.30 shows the acceptance check for reviewing and managing ETL automation controls from the portal.

Automation Control Management			
	Author:	Project Developer	
	Reviewer:	Project Supervisor / Company User	
Description: Verifies that an admin user can review and manage ETL automation controls from the portal.	Priority:	Medium	
	Test Type:	Functional / UAT	

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC15	Verifies that an admin user can review and manage ETL automation controls from the portal.	Admin user is logged in; automation control page is enabled.	Automation control page and selected schedule/control option.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Open Automation Control page.
2. Review current automation status.
3. Select an available management action.
4. Submit the action.
5. Confirm updated status/message.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The automation page displays current status and records the selected management action.

Actual Outcome

Automation status and control action were displayed clearly.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.30: TC15 Automation Control review

Figure 5.31 presents the test confirming that admin users can review and manage portal users according to role and status.

User Management Workflow			
		Author:	Project Developer
		Reviewer:	Project Supervisor / Company User
Description:	Verifies that an admin user can manage portal users according to assigned role and status.	Priority:	Medium
		Test Type:	Functional / UAT

Test Case ID	Objective	Preconditions	Input Data	Test Environment	Data Preparation
TC16	Verifies that an admin user can manage portal users according to assigned role and status.	Admin user is logged in.	Test user account data without real credentials.	Windows browser environment and deployed reporting portal	Use non-sensitive test account and selected report period prepared for validation

Test Steps

1. Open User Management page.
2. Review user list.
3. Open create/edit user form.
4. Enter non-sensitive test user details.
5. Save and review the updated user list.
6.
7.
8.

Postconditions

System state remains available for continued testing or user work.

Execution Date

2026-05-28

Expected Outcome

The user management page allows admin users to maintain user information safely.

Actual Outcome

User management workflow was completed using non-sensitive test data.

Bugs Encountered

1. No issues encountered.
2.
3.

Figure 5.31: TC16 User Management workflow

5.3.8 White-Box Testing

White-box testing was used to evaluate selected internal processing logic that could not be assessed fully through user-facing behaviour alone. In this project, white-box testing did not aim to provide exhaustive code-coverage measurement. Instead, it focused on the internal logic that had direct impact on report trustworthiness and operational safety, such as request-parameter validation, boundary-based refresh handling, source-to-replica quality-check flow, grouping and aggregation logic in report reconstruction, export-structure consistency, and role-restricted access handling.

This form of testing was relevant because a report can appear correct from the interface while still containing internal weaknesses in parameter interpretation, refresh-window replacement logic, or grouped output construction. By checking selected internal logic paths explicitly, the implementation could be reviewed not only at output level but also at process level. In academic terms, this strengthened the credibility of the completed system because it demonstrated that testing attention was

given to both visible behaviour and the underlying logic that produced that behaviour. The detailed white-box test cases are shown in Figures 5.32 to 5.40.

Figure 5.32 shows the internal validation checks applied to report parameters before backend execution proceeds.

WB01: API Parameter Validation Logic

White-box testing evidence. This layout focuses on internal logic checkpoints instead of user-facing UAT steps.

White-Box Test ID	WB01	**Module / Component**	Backend API router and request schema
Test Method	Boundary and invalid-input checks	**Risk Area**	Checks that backend report endpoints validate required filters before executing report logic.
Reviewer	Project developer	**Execution Date**	2026-05-29

Internal Logic Under Test:
Checks that backend report endpoints validate required filters before executing report logic.

This sheet records internal checkpoints, controlled conditions, and observed processing behaviour. It is not a user acceptance test case.

Controlled Inputs and Preconditions

Input / Condition	Expected Internal Handling	Observed Handling	Evidence Method	Result	Notes
Validated report period, role, or request payload depending on module	Processing should follow defined validation, grouping, refresh, or access-control path	Observed behaviour followed expected internal route during test scenario	Code-path review, API response inspection, and report-output cross-check	Passed	No credentials, server addresses, or proprietary setup details documented

Internal Checkpoint Trace

No.	Internal Checkpoint	Expected Internal Behaviour	Observed Evidence	Result
1	Required date parameter	Missing date should be rejected before query execution	Validation layer returned controlled error response	Passed
2	Date range order	Invalid start/end sequence should not produce misleading output	Invalid period was rejected or normalised by validation path	Passed
3	Outlet/report filter	Unexpected filter value should not break report execution	Controlled validation behaviour observed	Passed
4	Response consistency	Valid request should return structured report payload	Successful response followed expected JSON structure	Passed

Boundary / Negative Case Review

Boundary Case	Expected Protection	Observed Result	Status	Reviewer Note
Invalid or incomplete input	System should reject or handle safely without misleading report output	Controlled handling observed	Passed	No uncontrolled failure recorded
Unauthorised or inappropriate operation where relevant	System should restrict protected actions	Role/access guard remained effective in tested path	Passed	Applies to admin-related modules
Empty or incomplete data state	System should display understandable status rather than false result	Status/empty-state handling remained understandable	Passed	Supports operational safety

White-Box Test Conclusion

The selected internal logic path behaved as expected for the tested conditions. The test supports the implementation explanation in Chapter 5 without exposing internal credentials, server access instructions, or sensitive operational details.

Figure 5.32: WB01 API parameter validation

Figure 5.33 records the internal review of payment grouping and total-calculation logic in the Payment Type report path.

WB02: Payment Grouping and Total Calculation Logic

White-box testing evidence. This layout focuses on internal logic checkpoints instead of user-facing UAT steps.

White-Box Test ID	WB02	**Module / Component**	Payment Type report service/query layer
Test Method	Grouping and aggregation path review	**Risk Area**	Checks internal grouping and total logic for payment-oriented report reconstruction.
Reviewer	Project developer	**Execution Date**	2026-05-29

Internal Logic Under Test:
Checks internal grouping and total logic for payment-oriented report reconstruction.

This sheet records internal checkpoints, controlled conditions, and observed processing behaviour. It is not a user acceptance test case.

Controlled Inputs and Preconditions

Input / Condition	Expected Internal Handling	Observed Handling	Evidence Method	Result	Notes
Validated report period, role, or request payload depending on module	Processing should follow defined validation, grouping, refresh, or access-control path	Observed behaviour followed expected internal route during test scenario	Code-path review, API response inspection, and report-output cross-check	Passed	No credentials, server addresses, or proprietary setup details documented

Internal Checkpoint Trace

No.	Internal Checkpoint	Expected Internal Behaviour	Observed Evidence	Result
1	Grouping key	Payment rows should be grouped by the intended payment classification	Grouped output followed expected payment categories	Passed
2	Aggregation	Amounts should be summed at group and report-total level	Subtotal and total calculations aligned with validation evidence	Passed
3	Null/blank handling	Blank optional fields should not break grouping	Rows with optional blanks remained processable	Passed
4	Export mapping	Exported totals should match displayed totals	Displayed and exported totals aligned	Passed

Boundary / Negative Case Review

Boundary Case	Expected Protection	Observed Result	Status	Reviewer Note
Invalid or incomplete input	System should reject or handle safely without misleading report output	Controlled handling observed	Passed	No uncontrolled failure recorded
Unauthorised or inappropriate operation where relevant	System should restrict protected actions	Role/access guard remained effective in tested path	Passed	Applies to admin-related modules
Empty or incomplete data state	System should display understandable status rather than false result	Status/empty-state handling remained understandable	Passed	Supports operational safety

White-Box Test Conclusion

The selected internal logic path behaved as expected for the tested conditions. The test supports the implementation explanation in Chapter 5 without exposing internal credentials, server access instructions, or sensitive operational details.

Figure 5.33: WB02 Payment Type grouping logic

Figure 5.34 presents the rerun logic review used to confirm that repeated ETL execution does not duplicate reporting output.

WB03: ETL Re-run and Idempotency Logic

White-box testing evidence. This layout focuses on internal logic checkpoints instead of user-facing UAT steps.

White-Box Test ID	WB03	**Module / Component**	ETL scripts and reporting replica load flow
Test Method	Controlled rerun/checkpoint review	**Risk Area**	Checks that repeated ETL execution for the same reporting period does not create duplicated reporting output.
Reviewer	Project developer	**Execution Date**	2026-05-29

Internal Logic Under Test:
Checks that repeated ETL execution for the same reporting period does not create duplicated reporting output.

This sheet records internal checkpoints, controlled conditions, and observed processing behaviour. It is not a user acceptance test case.

Controlled Inputs and Preconditions

Input / Condition	Expected Internal Handling	Observed Handling	Evidence Method	Result	Notes
Validated report period, role, or request payload depending on module	Processing should follow defined validation, grouping, refresh, or access-control path	Observed behaviour followed expected internal route during test scenario	Code-path review, API response inspection, and report-output cross-check	Passed	No credentials, server addresses, or proprietary setup details documented

Internal Checkpoint Trace

No.	Internal Checkpoint	Expected Internal Behaviour	Observed Evidence	Result
1	Existing period load	Rerun should refresh or preserve target period without duplicate rows	Repeated run did not create duplicate report rows in validated period	Passed
2	Load boundary	ETL should apply selected period boundary consistently	Loaded records stayed within selected sync boundary	Passed
3	Failure handling	Failed step should not be treated as successful completion	Status remained traceable for failed/incomplete cases	Passed
4	Post-load reporting	Reports should read from refreshed replica state	Report output reflected refreshed period after completion	Passed

Boundary / Negative Case Review

Boundary Case	Expected Protection	Observed Result	Status	Reviewer Note
Invalid or incomplete input	System should reject or handle safely without misleading report output	Controlled handling observed	Passed	No uncontrolled failure recorded
Unauthorised or inappropriate operation where relevant	System should restrict protected actions	Role/access guard remained effective in tested path	Passed	Applies to admin-related modules
Empty or incomplete data state	System should display understandable status rather than false result	Status/empty-state handling remained understandable	Passed	Supports operational safety

White-Box Test Conclusion

The selected internal logic path behaved as expected for the tested conditions. The test supports the implementation explanation in Chapter 5 without exposing internal credentials, server access instructions, or sensitive operational details.

Figure 5.34: WB03 boundary-based ETL rerun

Figure 5.35 shows the internal checkpoint review for staged refresh windows and controlled historical completeness handling.

WB04: Staged Refresh Window Logic

White-box testing evidence. This layout focuses on internal logic checkpoints instead of user-facing UAT steps.

White-Box Test ID	WB04	**Module / Component**	Replica refresh and backfill process
Test Method	Refresh-boundary logic review	**Risk Area**	Checks that controlled refresh/backfill windows support historical completeness without uncontrolled production changes.
Reviewer	Project developer	**Execution Date**	2026-05-29

Internal Logic Under Test:
Checks that controlled refresh/backfill windows support historical completeness without uncontrolled production changes.

This sheet records internal checkpoints, controlled conditions, and observed processing behaviour. It is not a user acceptance test case.

Controlled Inputs and Preconditions

Input / Condition	Expected Internal Handling	Observed Handling	Evidence Method	Result	Notes
Validated report period, role, or request payload depending on module	Processing should follow defined validation, grouping, refresh, or access-control path	Observed behaviour followed expected internal route during test scenario	Code-path review, API response inspection, and report-output cross-check	Passed	No credentials, server addresses, or proprietary setup details documented

Internal Checkpoint Trace

No.	Internal Checkpoint	Expected Internal Behaviour	Observed Evidence	Result
1	Selected period	Refresh should target selected historical/current period	Refresh scope followed selected period boundary	Passed
2	Historical gaps	Known incomplete periods should remain identifiable	Quality check highlighted incomplete state where applicable	Passed
3	Operational control	Unattended changes should be managed through approved controls	Automation controls and manual sync path were separated clearly	Passed
4	Report readiness	Report validation should be performed after refresh	Validation was performed after refreshed data was available	Passed

Boundary / Negative Case Review

Boundary Case	Expected Protection	Observed Result	Status	Reviewer Note
Invalid or incomplete input	System should reject or handle safely without misleading report output	Controlled handling observed	Passed	No uncontrolled failure recorded
Unauthorised or inappropriate operation where relevant	System should restrict protected actions	Role/access guard remained effective in tested path	Passed	Applies to admin-related modules
Empty or incomplete data state	System should display understandable status rather than false result	Status/empty-state handling remained understandable	Passed	Supports operational safety

White-Box Test Conclusion

The selected internal logic path behaved as expected for the tested conditions. The test supports the implementation explanation in Chapter 5 without exposing internal credentials, server access instructions, or sensitive operational details.

Figure 5.35: WB04 staged refresh safety

Figure 5.36 records the internal logic checks used to detect completeness or consistency issues after replication.

WB05: Data Quality Detection Logic

White-box testing evidence. This layout focuses on internal logic checkpoints instead of user-facing UAT steps.

White-Box Test ID	WB05	**Module / Component**	Data quality check process
Test Method	Completeness and consistency checkpoints	**Risk Area**	Checks that the platform can identify data completeness issues after replication.
Reviewer	Project developer	**Execution Date**	2026-05-29

Internal Logic Under Test:
Checks that the platform can identify data completeness issues after replication.

This sheet records internal checkpoints, controlled conditions, and observed processing behaviour. It is not a user acceptance test case.

Controlled Inputs and Preconditions

Input / Condition	Expected Internal Handling	Observed Handling	Evidence Method	Result	Notes
Validated report period, role, or request payload depending on module	Processing should follow defined validation, grouping, refresh, or access-control path	Observed behaviour followed expected internal route during test scenario	Code-path review, API response inspection, and report-output cross-check	Passed	No credentials, server addresses, or proprietary setup details documented

Internal Checkpoint Trace

No.	Internal Checkpoint	Expected Internal Behaviour	Observed Evidence	Result
1	Completeness check	System should detect incomplete reporting period	Incomplete state was visible through quality-check result	Passed
2	Status display	Quality status should be understandable to admin user	Status was displayed in admin workflow	Passed
3	No credential exposure	Quality result should not expose server credentials	Only safe status information was shown	Passed
4	Follow-up action	Admin should know whether recheck/sync is needed	Result supported operational decision	Passed

Boundary / Negative Case Review

Boundary Case	Expected Protection	Observed Result	Status	Reviewer Note
Invalid or incomplete input	System should reject or handle safely without misleading report output	Controlled handling observed	Passed	No uncontrolled failure recorded
Unauthorised or inappropriate operation where relevant	System should restrict protected actions	Role/access guard remained effective in tested path	Passed	Applies to admin-related modules
Empty or incomplete data state	System should display understandable status rather than false result	Status/empty-state handling remained understandable	Passed	Supports operational safety

White-Box Test Conclusion

The selected internal logic path behaved as expected for the tested conditions. The test supports the implementation explanation in Chapter 5 without exposing internal credentials, server access instructions, or sensitive operational details.

Figure 5.36: WB05 source-to-replica data quality check

Figure 5.37 presents the internal workflow review from detected data issue through repair-related follow-up and recheck.

WB06: Data Quality Repair and Recheck Routing

White-box testing evidence. This layout focuses on internal logic checkpoints instead of user-facing UAT steps.

White-Box Test ID	WB06	**Module / Component**	Manual sync worker and data quality recheck flow
Test Method	Workflow checkpoint review	**Risk Area**	Checks the internal process flow from detected issue to repair/sync and recheck.
Reviewer	Project developer	**Execution Date**	2026-05-29

Internal Logic Under Test:
Checks the internal process flow from detected issue to repair/sync and recheck.

This sheet records internal checkpoints, controlled conditions, and observed processing behaviour. It is not a user acceptance test case.

Controlled Inputs and Preconditions

Input / Condition	Expected Internal Handling	Observed Handling	Evidence Method	Result	Notes
Validated report period, role, or request payload depending on module	Processing should follow defined validation, grouping, refresh, or access-control path	Observed behaviour followed expected internal route during test scenario	Code-path review, API response inspection, and report-output cross-check	Passed	No credentials, server addresses, or proprietary setup details documented

Internal Checkpoint Trace

No.	Internal Checkpoint	Expected Internal Behaviour	Observed Evidence	Result
1	Issue detection	Incomplete period should trigger recheck or sync action	Detected issue was routed to supported sync/recheck flow	Passed
2	Repair/sync action	Manual sync should be able to refresh selected data scope	Sync request processed through portal-supported path	Passed
3	Recheck	Quality status should update after data refresh	Recheck reflected refreshed data state	Passed
4	Report reuse	Reports should use updated replica data after recheck	Report validation used refreshed output	Passed

Boundary / Negative Case Review

Boundary Case	Expected Protection	Observed Result	Status	Reviewer Note
Invalid or incomplete input	System should reject or handle safely without misleading report output	Controlled handling observed	Passed	No uncontrolled failure recorded
Unauthorised or inappropriate operation where relevant	System should restrict protected actions	Role/access guard remained effective in tested path	Passed	Applies to admin-related modules
Empty or incomplete data state	System should display understandable status rather than false result	Status/empty-state handling remained understandable	Passed	Supports operational safety

White-Box Test Conclusion

The selected internal logic path behaved as expected for the tested conditions. The test supports the implementation explanation in Chapter 5 without exposing internal credentials, server access instructions, or sensitive operational details.

Figure 5.37: WB06 quality repair request flow

Figure 5.38 shows the internal review used to confirm consistency between displayed report data and exported output.

WB07: Report Export Mapping Logic

White-box testing evidence. This layout focuses on internal logic checkpoints instead of user-facing UAT steps.

White-Box Test ID	WB07	**Module / Component**	Frontend report export utility and report table model
Test Method	Display-to-export mapping review	**Risk Area**	Checks that displayed report data and exported report data remain consistent.
Reviewer	Project developer	**Execution Date**	2026-05-29

Internal Logic Under Test:
Checks that displayed report data and exported report data remain consistent.

This sheet records internal checkpoints, controlled conditions, and observed processing behaviour. It is not a user acceptance test case.

Controlled Inputs and Preconditions

Input / Condition	Expected Internal Handling	Observed Handling	Evidence Method	Result	Notes
Validated report period, role, or request payload depending on module	Processing should follow defined validation, grouping, refresh, or access-control path	Observed behaviour followed expected internal route during test scenario	Code-path review, API response inspection, and report-output cross-check	Passed	No credentials, server addresses, or proprietary setup details documented

Internal Checkpoint Trace

No.	Internal Checkpoint	Expected Internal Behaviour	Observed Evidence	Result
1	Visible columns	Export should include expected visible fields	Exported file retained report columns needed for review	Passed
2	Value consistency	Exported values should match table display	Exported values matched displayed report rows in sample checks	Passed
3	Totals	Export should not alter report totals	Exported totals aligned with on-screen totals where included	Passed
4	Formatting safety	Export should not include hidden credentials or internal paths	Export contained report data only	Passed

Boundary / Negative Case Review

Boundary Case	Expected Protection	Observed Result	Status	Reviewer Note
Invalid or incomplete input	System should reject or handle safely without misleading report output	Controlled handling observed	Passed	No uncontrolled failure recorded
Unauthorised or inappropriate operation where relevant	System should restrict protected actions	Role/access guard remained effective in tested path	Passed	Applies to admin-related modules
Empty or incomplete data state	System should display understandable status rather than false result	Status/empty-state handling remained understandable	Passed	Supports operational safety

White-Box Test Conclusion

The selected internal logic path behaved as expected for the tested conditions. The test supports the implementation explanation in Chapter 5 without exposing internal credentials, server access instructions, or sensitive operational details.

Figure 5.38: WB07 report export mapping

Figure 5.39 records the access-control logic review protecting administrative routes and backend operations.

WB08: Role-Based Access Guard Logic

White-box testing evidence. This layout focuses on internal logic checkpoints instead of user-facing UAT steps.

White-Box Test ID	WB08	**Module / Component**	Authentication/authorization layer and portal routing
Test Method	Access-control path review	**Risk Area**	Checks that route and backend access controls protect administrative features.
Reviewer	Project developer	**Execution Date**	2026-05-29

Internal Logic Under Test:
Checks that route and backend access controls protect administrative features.

This sheet records internal checkpoints, controlled conditions, and observed processing behaviour. It is not a user acceptance test case.

Controlled Inputs and Preconditions

Input / Condition	Expected Internal Handling	Observed Handling	Evidence Method	Result	Notes
Validated report period, role, or request payload depending on module	Processing should follow defined validation, grouping, refresh, or access-control path	Observed behaviour followed expected internal route during test scenario	Code-path review, API response inspection, and report-output cross-check	Passed	No credentials, server addresses, or proprietary setup details documented

Internal Checkpoint Trace

No.	Internal Checkpoint	Expected Internal Behaviour	Observed Evidence	Result
1	Normal user access	Normal user should not open admin pages	Admin-only routes were blocked for normal user scenario	Passed
2	Admin user access	Admin user should be able to open admin pages	Admin scenario could open admin pages	Passed
3	Backend protection	Protected operations should require authorized request	Admin operations followed protected access path	Passed
4	Navigation visibility	UI should not encourage unauthorised use	Admin navigation followed role-aware behaviour	Passed

Boundary / Negative Case Review

Boundary Case	Expected Protection	Observed Result	Status	Reviewer Note
Invalid or incomplete input	System should reject or handle safely without misleading report output	Controlled handling observed	Passed	No uncontrolled failure recorded
Unauthorised or inappropriate operation where relevant	System should restrict protected actions	Role/access guard remained effective in tested path	Passed	Applies to admin-related modules
Empty or incomplete data state	System should display understandable status rather than false result	Status/empty-state handling remained understandable	Passed	Supports operational safety

White-Box Test Conclusion

The selected internal logic path behaved as expected for the tested conditions. The test supports the implementation explanation in Chapter 5 without exposing internal credentials, server access instructions, or sensitive operational details.

Figure 5.39: WB08 role-based access restriction

Figure 5.40 presents the controlled request-handling review for automation-management actions in the admin workflow.

WB09: Automation-Control Request Handling

White-box testing evidence. This layout focuses on internal logic checkpoints instead of user-facing UAT steps.

White-Box Test ID	WB09	**Module / Component**	Automation control API and admin frontend page
Test Method	Request/status checkpoint review	**Risk Area**	Checks that automation management requests are handled through controlled admin workflow.
Reviewer	Project developer	**Execution Date**	2026-05-29

Internal Logic Under Test:
Checks that automation management requests are handled through controlled admin workflow.

This sheet records internal checkpoints, controlled conditions, and observed processing behaviour. It is not a user acceptance test case.

Controlled Inputs and Preconditions

Input / Condition	Expected Internal Handling	Observed Handling	Evidence Method	Result	Notes
Validated report period, role, or request payload depending on module	Processing should follow defined validation, grouping, refresh, or access-control path	Observed behaviour followed expected internal route during test scenario	Code-path review, API response inspection, and report-output cross-check	Passed	No credentials, server addresses, or proprietary setup details documented

Internal Checkpoint Trace

No.	Internal Checkpoint	Expected Internal Behaviour	Observed Evidence	Result
1	Status retrieval	Admin page should retrieve current automation status	Status was available for display	Passed
2	Action submission	Supported action should be sent as controlled request	Action request followed admin workflow	Passed
3	Response handling	User should receive understandable result	Success/status message shown after action	Passed
4	Safety boundary	Automation controls should not expose implementation secrets	Only safe operational status/control information was displayed	Passed

Boundary / Negative Case Review

Boundary Case	Expected Protection	Observed Result	Status	Reviewer Note
Invalid or incomplete input	System should reject or handle safely without misleading report output	Controlled handling observed	Passed	No uncontrolled failure recorded
Unauthorised or inappropriate operation where relevant	System should restrict protected actions	Role/access guard remained effective in tested path	Passed	Applies to admin-related modules
Empty or incomplete data state	System should display understandable status rather than false result	Status/empty-state handling remained understandable	Passed	Supports operational safety

White-Box Test Conclusion

The selected internal logic path behaved as expected for the tested conditions. The test supports the implementation explanation in Chapter 5 without exposing internal credentials, server access instructions, or sensitive operational details.

Figure 5.40: WB09 automation-control request handling

5.4 Summary

This chapter documented the implementation and testing of the Marrybrown Sales and Payment Analytics Platform. It described the completed development of the reporting-database and ETL layer, the FastAPI semantic/API layer, the React portal layer, and the supporting administrative functions that made the platform operationally

usable. It also outlined the formal report implementation coverage, the selected implementation examples, the recorded query-performance observation work, and the testing approach used to evaluate report accuracy, user-facing behaviour, and selected internal logic.

Taken together, the implementation and testing outcomes show that the project moved beyond conceptual design and delivered a real internal reporting platform with validated report modules, controlled refresh capability, role-based access, and company-requested extensions. The next chapter discusses the platform's contribution, remaining constraints, and future suggestions.